

Software Architecture & Design Principles

Software architecture is about the **big picture** and foundational structure of an application. It pertains to the things that must be **decided and agreed upon early** in the development process to make the software more maintainable, scalable, performant, and avoid a costly, difficult, and time-consuming restructuring later on. Software design is closely related but **concentrates on the code level design (or) "how" you will actually solve the architecture requirements.**

Core Objectives -

Correctness:

The system behaves as expected, integrates all components without conflicts, and satisfies the functional requirements specified by the stakeholders. It means-

- All modules/components work together coherently
- Interfaces and data flows between components are well-defined
- There are no logical contradictions or integration issues

[Example: Online Banking System.](#)

- Define separate services/modules.
- Each module has clearly defined APIs and communication contracts
- The Transaction Service only allows a fund transfer if:
 - a. The user is authenticated (checked via Authentication Service)
 - b. The account has sufficient balance (checked via Account Management Service)

Optimal Resource Consumption

- Minimal usage of computational resources without compromising performance
- Efficient algorithms, data structures, and communication patterns
- Consideration of hardware and deployment costs

[Example: Netflix](#)

Efficient Design:

- Adaptive bitrate streaming — video quality automatically adjusts based on internet speed.
- Implements caching for frequently accessed content to avoid repeated server requests.
- Uses CDNs to serve content closer to the user, reducing latency and bandwidth.

Benefits:

- Saves network bandwidth for both provider and user.
- Reduces server load and improves scalability.
- Lowers infrastructure cost while maintaining performance.

Inefficient Design:

- Streams the same high-resolution video to all users, regardless of network speed.
- No caching — every user request hits the origin server.
- Poorly managed memory leads to app crashes on low-end devices.

Modularity & Scalability

Modularity refers to dividing a software system into distinct, manageable, and independent components (or modules).

Design Goals:

1. High Cohesion: Related functions and data are grouped together.
2. Low Coupling: Modules know little about each other's internal workings. The best designs have **high cohesion** (also called strong cohesion) **within** a module and **low coupling** (also called weak coupling) **between** modules.
3. Separation of Concerns: Each module solves a clearly defined sub-problem.
4. Information Hiding: The information hidden can be data, data formats, behavior, and more generally, design decisions. When information is hidden there is an implied separation between interface and implementation. The information is hidden behind the **interface**.

Scalability means the system can handle increased load (data, users, features) without requiring a complete redesign.

Example: E-commerce Application

- User Management
- Product Catalog
- Shopping Cart
- Order Processing
- Payment Gateway
- Notification Service

Good Modularity Design:	Poor Modularity:
Product Catalog manages only product-related logic and data.	Product and cart logic are mixed in the same module.
Shopping Cart depends only on product IDs and quantities—not product details.	Modules access each other’s databases directly (tight coupling).
Each module exposes a simple API, like getProduct(id) or addItem(cart_id, product_id).	Adding new payment methods requires changing code in multiple modules.

Maintainability

How easily and efficiently a software system can be modified, extended, or corrected after it is deployed. A maintainable design supports:

- Adding new features
- Fixing bugs
- Improving performance
- Adapting to new technologies or user needs

Example: Blogging Platform - Poor Maintainability:

- All business logic is tightly coupled in a few large files
- Adding a new feature (e.g., “Like” button) requires touching comment, user, and post modules
- No clear documentation or modular structure

Key Principles of Software Architecture

The Principle Of Least Astonishment(POLA)

The principle of least astonishment (or Least Surprise) suggests that a solution or approach would not surprise a reasonably knowledgeable person in the subject area when encountered for the first time (the audience may vary e.g. end-user, programmer, tester, etc). In more practical terms, the principle aims to leverage the pre-existing knowledge of users to minimize their learning curve when using a module, so **anything with a high unpredictability factor is a good candidate for re-design**. It applies to every

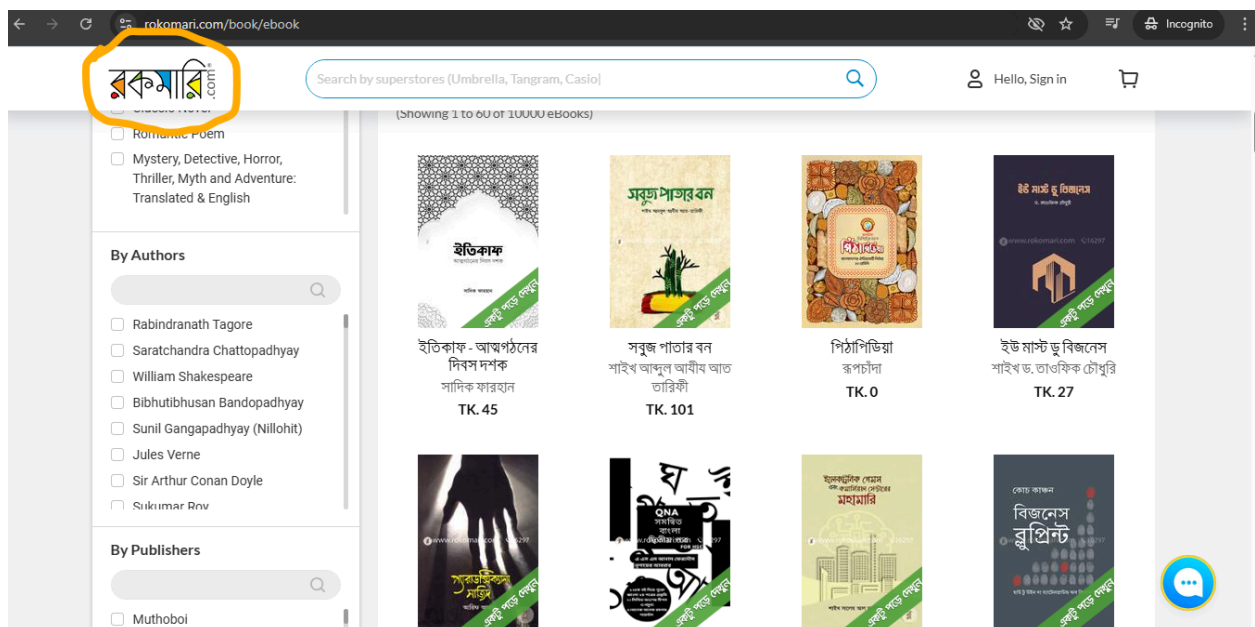
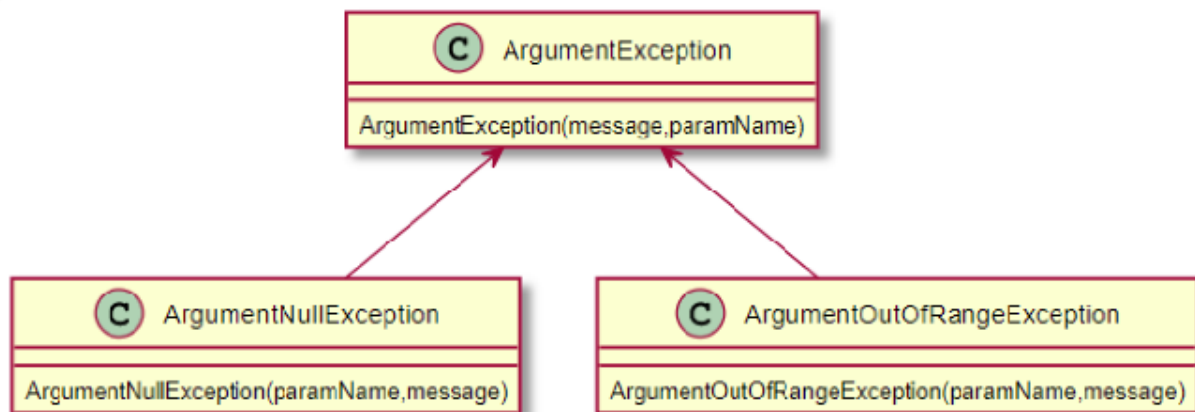
single aspect of the architecture: from naming services to the visualization of user interfaces, to the design of the domain model.

The POLA is probably more applicable during **UI design**, but is also relevant during software design.

In short, don't surprise the user (UI design) or programmer (software design) with unexpected behavior. Users and developers should be able to rely on their intuition.

Violation:

```
def square(x):  
    return x * 10 # Not actually squaring the number!
```



The Principle Of Least Effort(POLE)

This principle (also called Zipf's Law) stems from a basic human behavior: **Everyone tends to follow the path that is as close to effortless as possible.** So for example, if our design follows a particular pattern, the next developer will follow the same pattern again and again unless there is a significantly easier way to perform the task, in which case they will change! Or, taking this further, once they find acceptable results for a task, there is no immediate need to improve the current solution. Once we put the right design in place, we can create an architectural framework that will be the basis of the next systems we build. In other words, we are able to establish a successful and future-proof template for the organization's software systems.

Aspect	Developer Side	User Side
Goal	Stick with least effort pattern/code	Use features that are easiest to operate
POLE Effect	Reuse known code styles and templates	Avoid deep menus or complex interfaces

Example: driving learner license (<https://bsp.brt.gov.bd/?lan=en>)

The Principle Of Opportunity Cost(POOC)

This is what architecture is all about: weighing choices against each other and trying to make an informed decision on which one will add the most value for the project. For instance, a very common dichotomy is whether to create a tactical solution with quick time to market or a more strategic one which will be more expensive now with the view to leverage it in future projects and hence minimize the cost later down the line.

The Principle of Opportunity Cost states that **every architectural decision comes with trade-offs** — choosing one approach often means **forgoing another**, and your role is to **evaluate the long-term value vs. short-term gain.**

This principle pushes architects to ask:

- “What will we lose if we choose this path?”
- “Does this decision benefit the current project only, or can it be reused in the future?”

- “Is a quick fix today going to increase maintenance cost tomorrow?”

Example: API Design — Tactical vs. Strategic Choice

You’re building an e-commerce platform and need to expose product data via an API.

You have two options:

Option A: Tactical Solution (Quick & Dirty)

1. Build a hardcoded REST API specific to the current frontend project.
2. It works well and is delivered quickly.
3. But it’s not reusable for mobile apps or external integrations.

Opportunity Cost:

- You save time now, but will likely rewrite it later for other platforms.
- Limited scalability and flexibility.

Option B: Strategic Solution (Reusable & Extensible)

1. Design a RESTful service layer with a proper versioning and data abstraction.
2. Takes more time now.
3. But can serve multiple clients: web, mobile, partners, future tools.

Opportunity Cost:

- Delayed time to market for the current need.
- But saves effort and money later by avoiding redundant implementations

- POOC reminds architects: **you can’t choose everything** — but you should choose with awareness of what you're giving up.
- Making smart, context-aware decisions is what separates great software architecture from short-term patchwork.

The Principle Of Last Responsible Moment(POLRM)

This principle (aka **Cost of Delay**) originates from **Lean Software Development** and emphasizes holding on taking important actions and crucial decisions for as long as possible. This is done so as to not eliminate **important alternatives** until the last possible moment i.e. wait to narrow the options down until you are better informed.

delay irreversible decisions until the **latest point in time** when that decision **must be made** — and **not later**.

This approach allows:

- **More time to gather data**
- Better **understanding of risks, constraints, and user needs**
- Avoiding premature commitment that could lead to **expensive rework**

It's not procrastination — it's **strategic delay** to keep options open and make the **best-informed decision possible**.

Example: Choosing a Cloud Provider for a New App

You're developing a SaaS application and need to choose between AWS, Azure, or Google Cloud for deployment. But...

- You're still prototyping features
- You don't yet know your performance needs, traffic volume, or budget constraints
- Vendor-specific services (e.g., AWS Lambda vs. Azure Functions) might lock you in

Applying POLRM:

- **Instead of committing to a cloud vendor early on, you use containerization (e.g., Docker) and design the system.**
- You wait until your proof-of-concept is stable and you've gathered data on:
 - Expected user load
 - Cost estimates
 - Team expertise
- Only then do you choose the cloud provider that fits best

Basic **Software System Design** Principles (OOP Principles)

SOLID

Single Responsibility Principle

we can extend cohesion and say Every thing you write -- every module, class, interface, or method, should have one job. It should do the whole job and only that job. Similarly, it will have only one reason to change.

A class should have one and only one reason to change, meaning that a class should have only one job.

Why was it important to separate these two responsibilities into separate classes? Because each responsibility is an axis of change. When the requirements change, that change will be manifest through a change in responsibility amongst the classes. If a class assumes more than one responsibility, then there will be more than one reason for it to change.

If a class has more than one responsibility, then the **responsibilities become coupled**. Changes to one responsibility may impair or inhibit the ability of the class to meet the others. This kind of coupling leads to fragile designs that break in unexpected ways when changed.

Everything in the class should be related to that single purpose, i.e., be cohesive. It does not mean that your classes should only contain one method or property.

Example: <https://muatik.medium.com/oop-solid-with-examples-d3dc310d72c3>

```
public class Car {
    private double fuelLevel;

    public Car(double fuelLevel) {
        this.fuelLevel = fuelLevel;
    }

    public void drive(double distance) {
        if (this.fuelLevel >= distance) {
            this.fuelLevel -= distance;
            System.out.printf("Driving %.2f miles. Fuel level: %.2f\n",
                distance, this.fuelLevel);
        } else {
            System.out.println("Not enough fuel");
        }
    }

    public void fillGasTank(double amount) {
        this.fuelLevel += amount;
        System.out.printf("Adding %.2f gallons of gas. Fuel level: %.2f\n",
            amount, this.fuelLevel);
    }
}
```

To apply the Single Responsibility Principle, we can separate the responsibilities into their own classes. We can create a new class called FuelTank that is responsible for maintaining the fuel level.

```
public class Car {
    private FuelTank fuelTank;

    public Car(FuelTank fuelTank) {
        this.fuelTank = fuelTank;
    }

    public void drive(double distance) {
        if (this.fuelTank.hasEnoughFuel(distance)) {
            this.fuelTank.consumeFuel(distance);
            System.out.printf("Driving %.2f miles. Fuel level: %.2f\n",
distance, this.fuelTank.getFuelLevel());
        } else {
            System.out.println("Not enough fuel");
        }
    }
}

public class FuelTank {
    private double fuelLevel;

    public FuelTank(double fuelLevel) {
        this.fuelLevel = fuelLevel;
    }

    public boolean hasEnoughFuel(double distance) {
        return this.fuelLevel >= distance;
    }

    public void consumeFuel(double distance) {
        this.fuelLevel -= distance;
    }

    public double getFuelLevel() {
        return this.fuelLevel;
    }
}
```

Open-Closed Principle (OCP)

How can we create designs that are stable in the face of change and that will last longer than the first version? Bertrand Meyer gave us guidance as long ago as 1988 when he coined the now famous open-closed principle. To paraphrase him:

Software entities (classes, modules, functions, etc.) should be open for extension,
but closed for modification.

It should be possible to extend the function/behavior of a module without having to modify its code.

Example -

```
enum EmployeeType {
    Permanent, Contractual, PartTime
}

class Employee {
    private readonly EmployeeType type;
    private int transportAllowance;

    public Employee(EmployeeType type, int transportAllowance) {
        this.type = type;
        this.transportAllowance = transportAllowance;
    }

    public int CalculateSalary(int basic, int phr, int hw) {
        if (type == EmployeeType.Permanent) {
            return basic + basic / 4 + transportAllowance;
        } else if (type == EmployeeType.Contractual) {
            return basic + transportAllowance + phr * hw;
        } else {
            return basic + phr * hw;
        }
    }

    public int CalculateBonus(int basic) {
        if (type == EmployeeType.Permanent) {
            return basic / 2;
        } else if (type == EmployeeType.PartTime) {
            return 0;
        } else {
            return 5000;
        }
    }
}
```

How do you handle changing requirements and feedback that may affect the open-closed principle in OOP?

- ☐ Use abstraction and inheritance
- ☐ Use interfaces and polymorphism

```
enum EmployeeType {
    PERMANENT, CONTRACTUAL, PARTTIME
}

abstract class Employee {
    protected int transportAllowance;

    public Employee(int transportAllowance) {
        this.transportAllowance = transportAllowance;
    }

    public abstract int calculateSalary(int basic, int phr, int hw);

    public abstract int calculateBonus(int basic);
}

class PermanentEmployee extends Employee {

    public PermanentEmployee(int transportAllowance) {
        super(transportAllowance);
    }

    @Override
    public int calculateSalary(int basic, int phr, int hw) {
        return basic + basic / 4 + transportAllowance;
    }
}
```

```
        @Override
        public int calculateBonus(int basic) {
            return basic / 2;
        }
    }

class ContractualEmployee extends Employee {
    public ContractualEmployee(int transportAllowance) {
        super(transportAllowance);
    }
    @Override
    public int calculateSalary(int basic, int phr, int hw) {
        return basic + transportAllowance + phr * hw;
    }
    @Override
    public int calculateBonus(int basic) {
        return 5000;
    }
}

class PartTimeEmployee extends Employee {
    public PartTimeEmployee(int transportAllowance) {
        super(transportAllowance);
    }
}
```

```

    @Override
    public int calculateSalary(int basic, int phr, int hw) {
        return basic + phr * hw;
    }
    @Override
    public int calculateBonus(int basic) {
        return 0;
    }
}

public class Main {
    public static void main(String[] args) {
        Employee permanentEmployee = new PermanentEmployee(3000);
        Employee contractualEmployee = new ContractualEmployee(2000);
        Employee partTimeEmployee = new PartTimeEmployee(1000);

        // Example calculations
        System.out.println("Permanent Employee Salary: " +
            permanentEmployee.calculateSalary(50000, 200, 160));
        System.out.println("Permanent Employee Bonus: " +
            permanentEmployee.calculateBonus(50000));

        System.out.println("Contractual Employee Salary: " +
            contractualEmployee.calculateSalary(30000, 150, 100));
        System.out.println("Contractual Employee Bonus: " +
            contractualEmployee.calculateBonus(30000));

        System.out.println("Part-Time Employee Salary: " +
            partTimeEmployee.calculateSalary(20000, 100, 80));
        System.out.println("Part-Time Employee Bonus: " +
            partTimeEmployee.calculateBonus(20000));
    }
}

```

Liskov Substitution Principle

It is a principle on class hierarchies and inheritance. Simply, **inherited and superclass objects should be interchangeable.**

The principle states that objects of a superclass should be replaceable with objects of a subclass without affecting the correctness of the program. In simpler terms, if class B is a subclass of class A, then wherever A is expected, B should be able to fit in without causing any errors or unexpected behavior.

Example

Bob manages a fleet of vehicles for a delivery company. The company uses different types of vehicles, such as trucks, vans, and bicycles, depending on the delivery requirements. Bob expects all these vehicles to perform certain tasks like starting the engine, driving to the destination, and stopping.

Applying LSP:

- If Bob uses a truck or a van, he expects them to operate similarly in the context of deliveries (e.g., they both startEngine, drive, and stop).
- However, if someone gives Bob a bicycle that can't start an engine (because it doesn't have one), then it would violate the expectations Bob has from all vehicles in his fleet.
- In this analogy, if the Vehicle class represents the generic vehicle behavior, then Truck and Van would be subclasses that adhere to the LSP, while a Bicycle would violate it if it couldn't perform all expected actions.

```
class Vehicle {
    public void startEngine() {
        // Logic to start the engine
    }

    public void drive() {
        // Logic to drive
    }

    public void stop() {
        // Logic to stop
    }
}

class Truck extends Vehicle {
    // Inherits the startEngine, drive, and stop methods
}

class Bicycle extends Vehicle {
    @Override
    public void startEngine() {
        throw new UnsupportedOperationException("Bicycles don't have engines!");
    }
}
```

Solution:

```
abstract class Vehicle {
    public abstract void drive();
    public abstract void stop();
}

class MotorVehicle extends Vehicle {
    public void startEngine() {
        // Logic to start the engine
    }

    @Override
    public void drive() {
        // Logic to drive
    }

    @Override
    public void stop() {
        // Logic to stop
    }
}

class Truck extends MotorVehicle {
    // Inherits startEngine, drive, and stop methods
}

class Bicycle extends Vehicle {
    @Override
    public void drive() {
        // Logic to pedal the bicycle
    }

    @Override
    public void stop() {
        // Logic to stop the bicycle
    }
}
```

Balancing LSP with Other Principles

LSP complements other SOLID principles, especially the Open/Closed Principle (OCP), by ensuring that your classes can be extended with new functionality without altering existing code in a way that violates expected behavior. The Liskov Substitution Principle is essential for writing robust, maintainable object-oriented code. By ensuring that subclasses can be substituted for their parent classes without unexpected behavior, you create a more reliable and predictable codebase.

Interface Segregation Principle (ISP)

Clients should not be forced to depend on interfaces they do not use.

This principle is the first principle that applies to Interfaces instead of classes in SOLID and it is similar to the single responsibility principle. *It states that clients should not be forced to depend upon interface members they do not use.* This means that the interface should have the minimal set of methods required to ensure functionality and be limited to only one functionality.

For Example, If we create a Burger interface, we don't need to implement the cheese() method because cheese isn't available for every Burger type.

Another one,

Violation of ISP (Fat Interface) - OldPrinter is forced to implement methods it doesn't support.

```
class Machine:
    def print(self, document):
        pass

    def scan(self, document):
        pass

    def fax(self, document):
        pass

# OldPrinter can only print, but is forced to implement scan and fax
class OldPrinter(Machine):
    def print(self, document):
        print("Printing:", document)

    def scan(self, document):
        raise NotImplementedError("OldPrinter does not support scanning")

    def fax(self, document):
        raise NotImplementedError("OldPrinter does not support faxing")
```

Solution:

Break the interface into smaller, specific ones

```
class Printer:
```

```
    def print(self, document):  
        pass
```

```
class Scanner:
```

```
    def scan(self, document):  
        pass
```

```
class Fax:
```

```
    def fax(self, document):  
        pass
```

Only implements what it needs

```
class SimplePrinter(Printer):
```

```
    def print(self, document):  
        print("Printing:", document)
```

```
class MultiFunctionPrinter(Printer, Scanner, Fax):
```

```
    def print(self, document):  
        print("Printing:", document)
```

```
    def scan(self, document):  
        print("Scanning:", document)
```

```
    def fax(self, document):  
        print("Faxing:", document)
```

DEPENDENCY INVERSION PRINCIPLE

High Level Modules Should Not Depend Upon Low Level Modules. Both Should Depend Upon Abstractions.

Abstractions should not depend on details. Details should depend on abstractions.

Consider the implications of high level modules that depend upon low level modules. It is the high level modules that **contain the important policy decisions** and business models of an application. It is these models that contain the identity of the application. Yet, when these modules depend upon the lower level modules, then changes to the lower level modules can have direct effects upon them; and can force them to change.

Moreover, it is high-level modules that we want to be able to **reuse**. When high level modules depend upon low level modules, it becomes very difficult to reuse those high level modules in different contexts.

Violation of DIP:

```
class MySQLDatabase:
    def connect(self):
        print("Connected to MySQL database")

class App:
    def __init__(self):
        self.db = MySQLDatabase() # App depends directly on a low-level detail

    def start(self):
        self.db.connect()
```

Problem:

1. The App class directly depends on MySQLDatabase.
2. You can't easily switch to PostgreSQL or mock it for testing.

Solution:

```

# Define an abstraction (interface)
class Database:
    def connect(self):
        pass

# Low-level module implements the abstraction
class MySQLDatabase(Database):
    def connect(self):
        print("Connected to MySQL database")

# High-level module depends on the abstraction, not the concrete class
class App:
    def __init__(self, db: Database):
        self.db = db # Depends on abstraction

    def start(self):
        self.db.connect()

# Dependency injection: provide the actual implementation
db = MySQLDatabase()
app = App(db)
app.start()

```

DRY - Don't Repeat Yourself

Every piece of knowledge must have a single, unambiguous, authoritative representation within a system

A principle of software development aimed at reducing repetition of information which is likely to change, replacing it with abstractions that are less likely to change, or using data normalization which avoids redundancy in the first place.

Applying the DRY Principle in Practice

1. One way to implement the DRY principle is by utilizing methods. By **extracting commonly repeated code into methods**, we can centralize its logic and avoid duplicating it across our application. This approach allows for easier maintenance and promotes code reusability.
2. Another technique involves using classes and inheritance. We can **move duplicated code to a base class** and have other classes derive from it, ensuring that modifications only need to be made in one place.
3. Lastly, **interfaces** can be used to share code among classes without a common base. By defining an interface and implementing it in relevant classes, we can achieve code sharing while maintaining flexibility
4. Extract Constants or Configurations: If certain constants or configurations are repeated, centralize them to a single source to avoid redundancy.

Example:

Violation

```
def calculate_area_of_square(side):  
    return side * side  
  
def print_area_of_square(side):  
    area = side * side # Repeated Logic  
    print("Area is:", area)
```

Solution:

```
def calculate_area_of_square(side):  
    return side * side  
  
def print_area_of_square(side):  
    area = calculate_area_of_square(side)  
    print("Area is:", area)
```

DRY Applies Beyond Code:

- Database schema: Avoid duplicate data fields across tables
- Configuration files: Use variables or includes
- UI logic: Centralize button behavior rather than repeating logic in every screen

KISS- Keep it Simple, Stupid

The goal is to prioritize straightforward solutions over complex ones. By keeping our code simple, we can enhance comprehensibility, usability, and maintainability. The KISS principle applies not only to code but also to various aspects of software development, including architecture and user interface design.

Advantages of the KISS Principle

- Improved User Experience: Users prefer intuitive, easy-to-use interfaces and straightforward workflows. By embracing simplicity, we can provide a frictionless experience and meet user's expectations.
- Efficient Testing: Simpler software structures facilitate testing, including automated testing. With less complexity, testing becomes easier and more effective, leading to higher code quality.
- Easier Maintenance: A simpler codebase reduces complexity and makes maintenance and onboarding of new team members more efficient. By avoiding unnecessary features and adhering to coding standards, we can streamline the development process.

Violation of KISS (Overengineering)

```
def add(a, b):  
    # Complex and unnecessary abstraction  
    operations = {  
        'add': lambda x, y: x + y  
    }  
    return operations['add'](a, b)
```

```
def add(a, b):  
    return a + b
```

Following KISS

YAGNI - You Ain't Gonna Need It

Don't implement something until you actually need it.

It's a principle from Extreme Programming (XP) that helps avoid overengineering and keeps code clean, maintainable, and relevant to real requirements.

It reminds developers to implement functionality only when it is truly necessary. This principle encourages a minimalist approach to development, focusing on delivering the most critical features in a timely manner. By avoiding the implementation of unnecessary functionality, we can optimize development time, reduce complexity, and increase flexibility.

Implementing the YAGNI Principle

To apply the YAGNI principle effectively, it is essential to **prioritize** features based on their necessity. Focus on **delivering the core functionality** first, ensuring that it meets the requirements and provides value to users. Avoid adding features based on **speculation** or future needs that are not currently essential. Regularly reassess the project's requirements, gather feedback, and adapt accordingly. By following an iterative and user-centric development approach, we can ensure that we are delivering what is truly needed.

Example: You're building a user login system

```
class User
{
    private string _username;
    private string _password;
    private string _phoneNumber;    // Not needed
    private string _address;        // Not needed
    private bool _twoFactorEnabled; // No 2FA yet
    private string _profilePicture; // No UI yet

    public User(string username, string password)
    {
        _username = username;
        _password = password;
        _twoFactorEnabled = false; // Premature feature
    }
}
```

```

public bool Authenticate(string inputPassword)
{
    return _password == inputPassword;
}

// Premature methods for unused features
public void EnableTwoFactor() { _twoFactorEnabled = true; }
public void SetProfilePicture(string picturePath) { _profilePicture = picturePath; }
public void SetAddress(string address) { _address = address; }
public void SetPhoneNumber(string phone) { _phoneNumber = phone; }
}

```

Better approach:

```

class User
{
    private string _username;
    private string _password;

    public User(string username, string password)
    {
        _username = username;
        _password = password;
    }

    public bool Authenticate(string inputPassword)
    {
        return _password == inputPassword;
    }
}

```

Takeaway:

- Don't build for “**what if**”.
- Build for “**what is currently required**”.
- Add features **only when they're truly needed**.

References:

<https://www.geeksforgeeks.org/design-goals-and-principles-of-object-oriented-programming/>

https://www.youtube.com/watch?v=oCBA_EUysS8

<https://medium.com/@peterlee2068/software-design-principles-every-programmer-should-know-c164a83c6f87>

<https://www.linkedin.com/pulse/software-architecture-design-principles-venkatesu-punugupati/>

<https://medium.com/@curiousraj/the-principles-of-clean-code-dry-kiss-and-yagni-f973aa95fc4d>